

PML

UNIT II: Notes

*(Note: Python Programming Language has been used for codes and Google Colab/Jupyter/python installed on system as interfaces, for detailed example refer colab note book:
https://colab.research.google.com/drive/1GEV5cj3q4uFcs-z2PQ3VYD3T9LGffb7U#scrollTo=Z5GIOK_coN3n)*

Loading and Preprocessing Data with TensorFlow

The Data API, Chaining Transformation, Shuffling the data, Preprocessing the data, putting everything together, prefetching, using the data with tf.keras, TF record format, compressed TF record files.

I. The Data API and Chaining Transformation in TensorFlow

Introduction to TensorFlow Data API

The TensorFlow Data API provides a powerful and efficient way to load, preprocess, and transform data. It enables the creation of input pipelines that can handle large datasets and complex transformations in an efficient manner.

Key Concepts of the Data API

1. **Datasets:** Collections of data elements (e.g., arrays, tensors, files).
2. **Transformations:** Operations that process and manipulate data (e.g., mapping functions, filtering, batching).
3. **Iteration:** Efficiently iterating over data elements in a dataset.
4. **Combining Datasets:** Merging multiple datasets into one.

Creating Datasets

You can create datasets from various sources such as lists, tensors, and files using the `tf.data.Dataset` class.

Creating Datasets: You can create a dataset from various sources like lists, tensors, text files, or TFRecord files.

From a Python List:

```
import tensorflow as tf
```

```
dataset = tf.data.Dataset.from_tensor_slices([1, 2, 3, 4, 5])
```

From Tensors:

```
tensor_data = tf.constant([1, 2, 3, 4, 5])
```

```
dataset = tf.data.Dataset.from_tensor_slices(tensor_data)
```

From Text Files:

```
dataset = tf.data.TextLineDataset("file.txt")
```

(explanation in <https://chatgpt.com/c/6780af8d-ada0-800c-a859-29f3db2c1abf>)

From TFRecord Files:

```
dataset = tf.data.TFRecordDataset("file.tfrecords")
```

Transformations

Transformations are operations that modify the data within the dataset. The Data API provides methods like `map`, `filter`, `batch`, `shuffle`, and `prefetch` for applying these transformations.

1. **Map**: Applies a function to each element in the dataset.
2. **Filter**: Filters elements based on a condition.
3. **Batch**: Combines elements into batches.
4. **Shuffle**: Randomizes the order of elements.
5. **Prefetch**: Allows the data pipeline to fetch data in the background while the model is training.

Transformations with simple example

Let's use the list [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] to illustrate each transformation:

1. Map

Explanation: Applies a function to each element in the list.

- **Example:** Multiply each number by 2.
- **Result:** [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]

1. Map

Explanation: Applies a function to each element in the dataset.

Example: Multiply each number by 2.

```
python

import tensorflow as tf

# Create a dataset from the list
dataset = tf.data.Dataset.from_tensor_slices([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

# Apply the map transformation
mapped_dataset = dataset.map(lambda x: x * 2)

# Print the result
for item in mapped_dataset:
    print(item.numpy())
```

 Copy code

Output:

```
2
4
6
8
10
12
14
16
18
20
```

 Copy code

2. Filter

Explanation: Keeps only the elements that meet a certain condition.

- **Example:** Keep numbers greater than 10.
- **Result:** [12, 14, 16, 18, 20]

2. Filter

Explanation: Keeps only the elements that meet a certain condition.

Example: Keep numbers greater than 10.

python

 Copy code

```
# Apply the filter transformation
filtered_dataset = mapped_dataset.filter(lambda x: x > 10)

# Print the result
for item in filtered_dataset:
    print(item.numpy())
```

Output:

 Copy code

```
12
14
16
18
20
```



3. Batch

Explanation: Groups elements into smaller sets of a specified size.

- **Example:** Group into batches of size 2.
- **Result:** [[12, 14], [16, 18], [20]]

3. Batch

Explanation: Groups elements into smaller sets of a specified size.

Example: Group into batches of size 2.

python

 Copy code

```
# Apply the batch transformation
batched_dataset = filtered_dataset.batch(2)

# Print the result
for batch in batched_dataset:
    print(batch.numpy())
```

Output:

csharp

 Copy code

```
[12 14]
[16 18]
[20]
```

4. Shuffle

Explanation: Randomizes the order of elements.

- **Example:** Shuffle the elements of the list.
- **Result:** The order will vary each time; example result might be [[16, 18], [12, 14], [20]]

4. Shuffle

Explanation: Randomizes the order of elements.

Example: Shuffle the elements of the list.

python

 Copy code

```
# Apply the shuffle transformation
shuffled_dataset = filtered_dataset.shuffle(buffer_size=10)

# Print the result
for item in shuffled_dataset:
    print(item.numpy())
```

Output (The order will vary every time):

 Copy code

```
16
12
20
18
14
```



5. Prefetch

Explanation: Optimizes performance by preparing data while the model is training, reducing wait times.

- **Note:** Prefetching doesn't change the content of the data but improves efficiency.

5. Prefetch

Explanation: Optimizes performance by preparing data while the model is training.

Example: Add prefetch to prepare data one step ahead.

python

 Copy code

```
# Apply the prefetch transformation
prefetch_dataset = filtered_dataset.prefetch(buffer_size=tf.data.AUTOTUNE)

# Prefetch is mainly for performance optimization, so results will be the same
for item in prefetch_dataset:
    print(item.numpy())
```

Output:

 Copy code

```
12
14
16
18
20
```



Summary

- **Map:** Transforms each element.
- **Filter:** Selects elements based on conditions.
- **Batch:** Groups elements into batches.
- **Shuffle:** Randomizes element order.
- **Prefetch:** Speeds up data loading during training.

Chaining Transformations

Transformations can be chained together to create complex data pipelines. This is useful for performing multiple preprocessing steps in sequence.

Example:

```
# Create a dataset from the list
dataset = tf.data.Dataset.from_tensor_slices(data)
```

```
# Chain transformations: Normalize, Batch, and Shuffle
transformed_dataset = dataset.map(lambda x: tf.cast(x, tf.float32) / 50.0) \
    .batch(2) \
    .shuffle(buffer_size=5)

# Print transformed dataset
print("Transformed dataset:")
for batch in transformed_dataset:
    print(batch.numpy())
```

Iteration

Iteration over datasets can be done using Python's iteration constructs.

Example:

```
# Iterate over the dataset
for element in transformed_dataset:
    print(element.numpy())
```

Summary

- **Data API:** Efficiently handles data loading and preprocessing.
- **Transformations:** Operations to modify dataset elements.
- **Chaining:** Combining multiple transformations to create complex pipelines.
- **Iteration:** Efficiently iterate over dataset elements.

II. Shuffling and Preprocessing the Data in TensorFlow

Introduction

Shuffling and preprocessing are essential steps in preparing data for machine learning models. Shuffling ensures that the data is randomized, which helps in reducing bias and improving the generalization of the model. Preprocessing involves transforming raw data into a format that can be used by the model.

Shuffling Data

Shuffling data means randomizing the order of data points. This helps in ensuring that the model does not learn any inherent order in the data, which could lead to overfitting.

Example:

```
import tensorflow as tf
```

```
# Sample data
data = [1, 2, 3, 4, 5]

# Create a dataset from the list
dataset = tf.data.Dataset.from_tensor_slices(data)

# Shuffle the dataset
shuffled_dataset = dataset.shuffle(buffer_size=5)

# Print shuffled dataset
print("Shuffled dataset:")
for element in shuffled_dataset:
    print(element.numpy())
```

Preprocessing Data

Preprocessing involves various steps to transform raw data into a suitable format for model training. Common preprocessing tasks include normalization, standardization, data augmentation, and tokenization.

Normalization

Normalization scales the data to a range, typically [0, 1].

Example:

```
# Normalize the data: scaling to [0, 1]
normalized_dataset = dataset.map(lambda x: tf.cast(x, tf.float32) / 5.0)

# Print normalized dataset
print("Normalized dataset:")
for element in normalized_dataset:
    print(element.numpy())
```

Standardization

Standardization transforms data to have a mean of 0 and a standard deviation of 1.

Example:

```
# Sample data
data = [1, 2, 3, 4, 5]

# Calculate mean and standard deviation
```

```
mean = tf.reduce_mean(data)
std_dev = tf.math.reduce_std(data)

# Create a dataset from the list
dataset = tf.data.Dataset.from_tensor_slices(data)

# Standardize the data
standardized_dataset = dataset.map(lambda x: (x - mean) / std_dev)

# Print standardized dataset
print("Standardized dataset:")
for element in standardized_dataset:
    print(element.numpy())
```

Data Augmentation (for images)

Data augmentation is used to create more training examples by applying random transformations to existing data.

Example:

```
import tensorflow as tf

# Sample data: assume images are 28x28 grayscale images
data = tf.random.uniform([5, 28, 28, 1], minval=0, maxval=255, dtype=tf.float32)

# Create a dataset from the images
dataset = tf.data.Dataset.from_tensor_slices(data)

# Apply data augmentation: random flip
augmented_dataset = dataset.map(lambda x: tf.image.random_flip_left_right(x))

# Print augmented dataset shapes
print("Augmented dataset shapes:")
for element in augmented_dataset:
    print(element.shape)
```

Tokenization (for text)

Tokenization is the process of converting text into sequences of tokens (words or subwords).

Example:

```
import tensorflow as tf

# Sample data
data = ["Hello TensorFlow", "Data preprocessing is important"]

# Create a tokenizer
tokenizer = tf.keras.preprocessing.text.Tokenizer()

# Fit the tokenizer on the data
tokenizer.fit_on_texts(data)

# Convert text to sequences of tokens
tokenized_data = tokenizer.texts_to_sequences(data)

# Print tokenized data
print("Tokenized data:")
print(tokenized_data)
```

Summary

- **Shuffling:** Randomizes the order of data points to reduce bias.
- **Normalization:** Scales data to a range, typically [0, 1].
- **Standardization:** Transforms data to have a mean of 0 and a standard deviation of 1.
- **Data Augmentation:** Applies random transformations to create more training examples (commonly used in image processing).
- **Tokenization:** Converts text into sequences of tokens.

III. Putting everything together and prefetching

Putting everything together-Combining Datasets You can combine multiple datasets using methods like concatenate and zip.

Example:

Combining Datasets:

The Data API allows combining multiple datasets to create more complex data pipelines. The `tf.data.Dataset` class provides methods like `concatenate` and `zip` to combine datasets.

```
import tensorflow as tf

# Sample data for two datasets
data1 = [[1, 2], [3, 4]]
data2 = [[5, 6], [7, 8]]

# Create datasets from the lists
dataset1 = tf.data.Dataset.from_tensor_slices(data1)
dataset2 = tf.data.Dataset.from_tensor_slices(data2)

# Concatenate the datasets
combined_dataset = dataset1.concatenate(dataset2)

# Print combined dataset
print("Combined dataset:")
for element in combined_dataset:
    print(element.numpy())

# Zip the datasets (assuming they have the same length)
zipped_dataset = tf.data.Dataset.zip((dataset1, dataset2))

# Print zipped dataset
print("\nZipped dataset:")
for (element1, element2) in zipped_dataset:
    print(f"Dataset 1: {element1.numpy()}, Dataset 2: {element2.numpy()}")

Combined dataset:
[1 2]
[3 4]
[5 6]
[7 8]

Zipped dataset:
Dataset 1: [1 2], Dataset 2: [5 6]
Dataset 1: [3 4], Dataset 2: [7 8]
```

prefetch Transformation

The `prefetch` method overlaps the preprocessing and model execution to improve performance.

Prefetching:

Example: In video streaming applications, prefetching can be used to load the next set of video frames while the current frames are being processed and displayed. This ensures smooth playback without noticeable delays between frames.

```
import tensorflow as tf
# Create a dataset from a list
dataset = tf.data.Dataset.from_tensor_slices([1, 2, 3, 4, 5])
# Apply the map transformation
dataset = dataset.map(lambda x: x * x)
# Apply the batch transformation with a batch size of 2
dataset = dataset.batch(2)
# Apply the prefetch transformation
#The prefetch method in TensorFlow's tf.data.Dataset API
#allows you to overlap the preprocessing of your data with
#model execution, improving performance by keeping the GPU/CPU
#busy while the data is being prepared.
#The buffer_size=tf.data.experimental.AUTOTUNE argument
#dynamically tunes the prefetch buffer size for optimal performance.
dataset = dataset.prefetch(buffer_size=tf.data.experimental.AUTOTUNE)
print("Prefetched Dataset:")
for batch in dataset:
    print(batch.numpy())
Prefetched Dataset:
[1 4]
[ 9 16]
[25]
```

IV. Using data with tf.keras

1. Introduction to tf.keras

- **Overview of tf.keras:** **tf.keras** is a high-level API in TensorFlow designed for building and training neural networks. It provides a user-friendly interface for creating complex models with minimal code.
- **Benefits of tf.keras:**
 - Simplifies model building and training.
 - Facilitates rapid prototyping and experimentation.

- Integrates seamlessly with TensorFlow's powerful features.

2. Data Preparation

- **Importance of Data Preprocessing:** Proper data preprocessing is crucial for improving model performance and ensuring accurate predictions.

Using **tf.keras.datasets** to Load Standard Datasets:

Code:

```
from tensorflow.keras.datasets import mnist

# Load the MNIST dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Print dataset shapes
print("Training data shape:", x_train.shape)
print("Test data shape:", x_test.shape)
```

- **Explanation:**

- `mnist.load_data()` downloads and splits the MNIST dataset into training and test sets.
- The shapes of the data are printed to understand the dimensions of the dataset.

Simple Data Exploration and Visualization:

Code:

```
import matplotlib.pyplot as plt

# Display the first image from the training set
plt.imshow(x_train[0], cmap='gray')
plt.title(f"Label: {y_train[0]}")
plt.show()
```

- **Explanation:**

- `plt.imshow()` visualizes the first image in the dataset.
- The label of the image is shown to understand what the image represents.

3. Ways to Use Data with **tf.keras**

a. Using NumPy Arrays

Directly Feeding NumPy Arrays into **tf.keras** Models:

Code:

```
import tensorflow as tf
import numpy as np
```

```
# Create dummy data (e.g., 100 grayscale images of size 28x28)
x_train = np.random.random((100, 28, 28)) # 100 images, random pixel values
y_train = np.random.randint(0, 10, 100) # 100 random labels (0 to 9)

# Define a simple model
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)), # Flatten 28x28 images into a 1D vector
    tf.keras.layers.Dense(128, activation='relu'), # Fully connected layer
    tf.keras.layers.Dense(10, activation='softmax') # Output layer for 10 classes
])

# Compile the model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Train the model
model.fit(x_train, y_train, epochs=5)
```

- **Explanation:**

- A model is built with layers to process and classify images.
- The model is compiled and trained using the MNIST dataset.

b. Using TensorFlow Datasets (TFDS)**Integrating with TensorFlow Datasets:****Code:**

```
import tensorflow_datasets as tfds

# Load MNIST dataset using TFDS
ds_train, ds_info = tfds.load('mnist', split='train', with_info=True)

# Prepare dataset
ds_train = ds_train.shuffle(ds_info.splits['train'].num_examples)
ds_train = ds_train.batch(32).prefetch(tf.data.experimental.AUTOTUNE)
```

- **Explanation:**

- `tfds.load()` loads the dataset and provides metadata.
- `shuffle`, `batch`, and `prefetch` are used for efficient data handling.

c. Data Augmentation**Enhancing Dataset Variability:****Code:**

```
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.experimental.preprocessing.RandomFlip('horizontal'),
    tf.keras.layers.experimental.preprocessing.RandomRotation(0.2),
])
```

```
# Define a model with data augmentation
```

```
model = tf.keras.Sequential([
    data_augmentation,
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])
```

- **Explanation:**

- Data augmentation techniques such as random flipping and rotation are applied to increase dataset variability.

d. Custom Data Input**Handling Custom Data Formats:****Code:**

```
# Load CSV files into a dataset
```

```
filenames = ['/path/to/data/file1.csv', '/path/to/data/file2.csv']
```

```
dataset = tf.data.experimental.make_csv_dataset(filenames, batch_size=32)
```

- **Explanation:**

- `tf.data.experimental.make_csv_dataset()` creates a dataset from CSV files, enabling custom data handling.

e. Transfer Learning**Using Pre-trained Models:****Code:**

```
# Load a pre-trained model
```

```
base_model = tf.keras.applications.MobileNetV2(weights='imagenet', include_top=False)
```

Define a new model on top of the pre-trained model

```
model = tf.keras.Sequential([  
    base_model,  
    tf.keras.layers.GlobalAveragePooling2D(),  
    tf.keras.layers.Dense(10, activation='softmax')  
])
```

- **Explanation:**

- MobileNetV2 is used as a base model for feature extraction.
- A new classification head is added for specific tasks.

4. Model Training and Evaluation

Compiling and Training a `tf.keras` Model:

Code:

```
# Compile the model  
model.compile(optimizer='adam',  
              loss='sparse_categorical_crossentropy',  
              metrics=['accuracy'])
```

Train the model

```
model.fit(ds_train, epochs=5, validation_data=ds_test)
```

- **Explanation:**

- The model is compiled and trained with the dataset, using validation data to monitor performance.

5. Model Deployment and Serving

Exporting Models for Deployment:

Code:

```
# Save the model  
model.save('path/to/save/model')
```

Load the model

```
loaded_model = tf.keras.models.load_model('path/to/saved/model')
```

- **Explanation:**

- `model.save()` exports the model for deployment.
- `tf.keras.models.load_model()` loads the saved model for inference.

6. Conclusion and Recap

- **Summary:**

- Overview of different methods for handling and preparing data with `tf.keras`.
- Discuss practical applications and considerations based on the data and problem domain.

V. TF record format and compressed TF record files

TFRecord Format

Purpose: The TFRecord format is a simple format for storing a sequence of binary records. It is used primarily in TensorFlow to store large datasets efficiently. TFRecord files are designed to be used with TensorFlow's data input pipeline, providing several benefits:

- **Efficiency:** TFRecord files are compact and can store large amounts of data, which helps in reducing storage space and improving I/O performance.
- **Flexibility:** They can store any type of data (images, text, etc.) in a binary format, which is more efficient for storage and processing.
- **Integration:** TFRecord files integrate seamlessly with TensorFlow's `tf.data` API, making it easier to create data input pipelines.

Example of Use:

Use Case: Storing a dataset of images and labels for training a neural network.

Suppose you have a dataset of images and corresponding labels that you want to use to train a convolutional neural network (CNN). Here's a brief overview of how you would use the TFRecord format:

1. **Convert Images and Labels to TFRecords:**
 - Read the images and labels from their source (e.g., JPEG files and a CSV file).
 - Create `tf.train.Example` objects for each image-label pair, which will be serialized into TFRecord format.
 - Write the serialized `tf.train.Example` objects to a TFRecord file.
2. **Create a Data Input Pipeline:**
 - Use TensorFlow's `tf.data.TFRecordDataset` to read the TFRecord file.
 - Parse the serialized data back into image and label tensors.
 - Perform any necessary preprocessing (e.g., normalization, data augmentation).
 - Batch and shuffle the data for training.

Creating a TF Record File

Steps to Create a TF Record File:

1. **Prepare Data:**
 - Imagine we have images and labels.
2. **Write Data to TF Record File:**
 - Use Python to convert images and labels into TF Record format.

Simple Code Example:

```
import tensorflow as tf
```

Function to create a TF Record

```
def create_tf_record(image_paths, labels, output_file):  
    with tf.io.TFRecordWriter(output_file) as writer:  
        for image_path, label in zip(image_paths, labels):  
            # Read image  
            image = open(image_path, 'rb').read()  
            # Create a feature  
            feature = {  
                'image': tf.train.Feature(bytes_list=tf.train.BytesList(value=[image])),  
                'label': tf.train.Feature(int64_list=tf.train.Int64List(value=[label])),  
            }  
            # Create an example  
            example = tf.train.Example(features=tf.train.Features(feature=feature))  
            # Write the example  
            writer.write(example.SerializeToString())
```

Example usage

```
create_tf_record(['image1.jpg', 'image2.jpg'], [0, 1], 'data.tfrecord')
```

Explanation:

- `tf.io.TFRecordWriter`: Opens a file to write TF Record data.
- `tf.train.Example`: Packs the data into a format TensorFlow can use.
- `writer.write()`: Writes the data to the TF Record file.

4. Compressed TF Record Files

Why Compress?

- Compression reduces the file size, which helps in saving storage space and can speed up data transfer.

How to Compress:

- TF Record files can be compressed using gzip or other compression formats.

Simple Code Example:

```
def create_compressed_tf_record(image_paths, labels, output_file):  
    with tf.io.TFRecordWriter(output_file, options='GZIP') as writer:
```

```
for image_path, label in zip(image_paths, labels):
    image = open(image_path, 'rb').read()
    feature = {
        'image': tf.train.Feature(bytes_list=tf.train.BytesList(value=[image])),
        'label': tf.train.Feature(int64_list=tf.train.Int64List(value=[label])),
    }
    example = tf.train.Example(features=tf.train.Features(feature=feature))
    writer.write(example.SerializeToString())
```

Example usage

```
create_compressed_tf_record(['image1.jpg', 'image2.jpg'], [0, 1], 'data_compressed.tfrecord.gz')
```

Explanation:

- `options='GZIP'`: Specifies that the file should be compressed using gzip.